

Statistical Signal Analysis Higgs Boson Modeling

Richard Khoo

December 2023

1 Signal Yield

1.1 Log-Likelihood Maximization

In the calculation of the signal yield, using the log-likelihood maximization was an important part of being able to fit the 4th-order polynomial to the data. We started with defining the 4th-order polynomial:

```
def poly4(myy,c):  
    return 1 + c[0]*myy + c[1]*myy**2 + c[2]*myy**3 + c[3]*myy**4
```

1.2 Fitting the Model to the Data

From the 4th-order polynomial, we were able to fit the data by using the minimize function from Scipy:

```
from scipy.optimize import minimize  
result = minimize(NLLpoly,x0=[-10,100,1,1,1],args=(obs),method='Nelder-Mead')
```

1.3 Extracting the Signal Yield

The total signal yield is extracted by taking the `c3[5]` parameter after minimizing `NLLsig_plus_bkg`, this parameter represents the best estimate for the number of signal events and also the 5th coefficient of the polynomial of the background model.

1.4 Significance of the Signal Yield

The signal yield allows us to measure the presence of a signal over the expected background.

2 Significance

2.1 How we got the P-Value

To find the P-Value of the data, we had to determine the amount of pseudo experiments that were generated under the background-only hypothesis that had a LLR value greater than the observed LLR value. From here the P-Value is just the ratio of the greater-than number and the length of the LLR_B array which represents the total number of pseudo experiments generated under the b-only hypothesis.

The code for this can be seen below:

```
greater_than = np.sum(LLR_B > (NLLR*-1))
pvalue = greater_than/len(LLR_B)
```

2.2 Significance of the P-Value

Our P-Value of 0.537 means that our observed value (observed NLLR) is very common under the background-only hypothesis. The corresponding Z-score of 0.09 means that the observed result is very close to the mean which means that it's not statistically significant.

3 Mass Measurement

3.1 Modified NLL function

We modified our NLL function by adding an variable "mH". We also change the mean of our signal PDF into mH and fix the sigma as 1.6. The modified NLL function for mass measurement are as follows:

```
def NLLSpluspolyfree(c, obs, mH):
    exp = poly4(myy, [c[0], c[1], c[2], c[3]]) * c[4] + sigpdf(myy, mH, 1.6) * c[5]
    NLLvalue = -1 * poisson.logpmf(obs, exp).sum()
    return NLLvalue
```

3.2 Minimizing NLL_mH

Following the same process as the number of the signal events, we conduct a scan over range (120,130) for the mH value. Then the mH value with the minimized

NLL yields the optimal mass of the Higgs boson.

3.3 68% confidence interval

As we get the minimized NLL value, we get the mH interval with 0.5 difference in NLL_ values by:

```
interval = x[np.where( NLL_values - min_NLL - 0.5 < 1e-20)]  
mH_left_bound = interval[0]  
mH_right_bound = interval[-1]
```

to get the left and right bounds.

The sigma errors are -0.14 and 0.15, which means our error for mass measurement is about 0.3eV. This error is not very significant compared to the particle mass.

4 Sonification of the Higgs Boson

4.1 The Signal Definition in the Time Domain

The signal definition in the time domain is stated here as a sine wave, it is calculated for each frequency in the array 'frequencies'. The code for this portion is shown below:

```
for freq in frequencies:  
    t = np.linspace(0, note_duration, int(rate * note_duration), endpoint=False)  
    sine_wave = np.sin(2 * np.pi * freq * t)  
    scaled_wave = np.int16(sine_wave / np.max(np.abs(sine_wave))) * 32767  
    f.writeframes(scaled_wave.tobytes())
```

In the first line:

```
t = np.linspace(0, note_duration, int(rate * note_duration), endpoint=False)
```

We have the creating of a time array for each note with the given rate and note duration, in this case 44100Hz and 0.05 sec per note.

The second and third line:

```
sine_wave = np.sin(2 * np.pi * freq * t)  
scaled_wave = np.int16(sine_wave / np.max(np.abs(sine_wave))) * 32767)
```

This code simply computes the sine wave for each frequency over the time array 't' and then scales it with the provided code.

4.2 Frequency Components in the Signal

In the code we have an option for 2 different frequencies to choose from, C Major, and A Harmonic Minor. This is done in the `data_to_minor_scale_frequencies` function as seen below:

```
def data_to_major_scale_frequencies(data, frequencies):
    scale_notes = list(frequencies.values())
    indices = np.round(data * (len(scale_notes) - 1)).astype(int)
    return [scale_notes[i] for i in indices]

# C Major Frequencies
C_major_frequencies = {
    'C': 261.63,
    'D': 293.66,
    'E': 329.63,
    'F': 349.23,
    'G': 392.00,
    'A': 440.00,
    'B': 493.88
}

# A Harmonic Minor Frequencies
A_harmonic_minor_frequencies = {
    'A': 440.00,
    'B': 493.88,
    'C': 523.25,
    'D': 587.33,
    'E': 659.26,
    'F': 698.46,
    'G#': 830.61,
}
```

`data_to_major_scale_frequencies` takes in 2 arrays, `data`: which is an array of normalized values which in this case is `normalized_residuals` and `frequencies` which is a dictionary of frequencies that represents a scale. `scale_notes = list(frequencies.values())` extracts the frequencies from the dictionary and puts them into a list (so that it's easy to add more scales) `indices = np.round(data`

* (len(scale_notes) - 1)).astype(int) This scales the normalized data to a range that matches the indices of scale_notes return [scale_notes[i] for i in indices] this iterates over each int in indices and uses it to select the corresponding frequency from the scale_notes list

4.3 Effects of Filtering

We choose the Butterworth low-pass filter to apply to our frequencies the code for it is below:

```
def butter_lowpass(cutoff, fs, order=5):
    nyq = 0.5 * fs
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    return b, a

def butter_lowpass_filter(data, cutoff, fs, order=5):
    b, a = butter_lowpass(cutoff, fs, order=order)
    y = lfilter(b, a, data)
    return y
```

These are the functions for a low pass Butterworth filter which creates a smooth frequency response in the passband, which is the range of frequencies up to the cutoff and then tapers off the stopband to 0 which are the frequencies above the designated cutoff frequencies.

`butter_lowpass(cutoff, fs, order=5):`

First starts off by calculating the Nyquist frequency, this is the theoretical limit for the highest frequency that can be sampled, then calculates the normal_cutoff which normalizes the cutoff frequency with respect to the Nyq frequency. Then `b, a = butter(terms)` generates the filter coefficients (b and a) to be used in the next function

`butter_lowpass_filter(data, cutoff, fs, order=5):`

This obtains the coefficients from the `butter_lowpass` function then applies the filter to the data. `lfilter(b, a, data)` filters data along one-dimension.

Without filtering our signal would contain every single frequency in the natural array, which creates a very unpleasant sound that doesn't sound good to the ear with these filters at least they follow a sort of diatonic scale system.